

实验报告：基于多维物理先验与图割优化的前景分割算法 (FSCut)

2300013097

信息科学技术学院

魏靖弢

1. 实验背景与动机 (Introduction)

- 本实验旨在解决 Weizmann2 数据集中的前景分割问题。由于该数据集样本量极小，直接应用现代深度学习模型（如 U-Net 或 Mask R-CNN）极易陷入过拟合。
- 为此，本研究摒弃了纯数据驱动的“黑盒”模型，设计并实现了一种基于**白盒数学物理先验**的全自动无监督分割框架——**FSCut (Felzenszwalb-Saliency Cut)**。实验过程经历了多次核心算法迭代，通过不断引入中心先验、空间上下文拓扑以及抗收缩膨胀力，逐步逼近物理真实的分割边界，最终将平均 F1-Score 从 0.618 提升至 0.702。

2. 算法迭代演进记录 (Methodology & Evolution)

2.0 版本零：正式代码前的思考与探索

- 观察EFG_BPP的算法，代码中未使用SLIC等算法进行超像素分割；参考之前比较主流的Fast_RCNN模型中的办法，我尝试使用Felzenszwalb办法。这个办法的速度更快，而且在图像分割的任务上，似乎有更好的表现。
 - SLIC 的特点（形状规则）：它是基于 K-Means 的，生成的超像素像蜂窝一样，大小均匀、形状规整。缺点是，遇到马的细长腿或者尾巴时，网格容易把细小结构给“切断”。
 - Felzenszwalb 的特点（顺应边缘）：它是基于最小生成树的，纯粹靠颜色梯度和边缘来切分。它生成的超像素极其不规则，完全顺应物体的真实物理边缘。

图割优化（Graph Cut）核心原理与能量最小化推导

- 受之前学习的《离散数学基础》课程启发，核心的计算逻辑是使用图的最大流最小割算法，寻找连通域作为单一物体，不同的连通域算作不同物体。

```

# =====
# 5. 构建图割 (Graph Cut)
# =====
rag = graph.rag_mean_color(rgb_img, segments)
g = maxflow.Graph[float]()
nodes = g.add_nodes(num_nodes)

# 添加数据项边
for i in range(num_nodes):
    sal_val = node_saliency[i]

    if i in boundary_segments:
        # 依然保留边界背景先验，稳住阵脚
        weight_fg, weight_bg = 0.0, 1000.0
    elif sal_val > otsu_thresh * 1.1:
        weight_fg, weight_bg = 1000.0, 0.0
    elif sal_val < otsu_thresh * 0.8:
        weight_fg, weight_bg = 0.0, 1000.0
    else:
        weight_fg = sal_val * 10.0
        weight_bg = (1.0 - sal_val) * 10.0

    g.add_tedge(nodes[i], weight_fg, weight_bg)

# 添加平滑项边
gamma = 900.0
lambda_smooth = 1.0
for edge in rag.edges:
    n1, n2 = edge
    color_diff = rag[n1][n2]['weight']
    smooth_weight = lambda_smooth * np.exp(-(color_diff ** 2) / gamma)
    g.add_edge(nodes[n1], nodes[n2], smooth_weight, smooth_weight)

# 求解最大流最小割
g.maxflow()
binary_mask = np.zeros_like(segments, dtype=np.uint8)
for i in range(num_nodes):
    if g.get_segment(nodes[i]) == 0:
        binary_mask[segments == i] = 1

```

我们将图像（或超像素块）抽象为一个带权无向图 $G = (V, E)$ ，并通过求解该图的“最大流最小割”来获得全局最优的物理边界。

图割算法的核心目标是最小化以下马尔可夫随机场（MRF）的能量泛函：

$$E(L) = \sum_{p \in V} D_p(L_p) + \sum_{(p,q) \in N} V_{p,q}(L_p, L_q)$$

其中, $L_p \in \{0, 1\}$ 表示超像素块 p 的标签 (前景或背景)。公式由两部分组成, 它们在代码中分别对应了 **数据项边 (T-links)** 和 **平滑项边 (N-links)** 的构建。

2.1 数据项 (Data Term, D_p): T-links 终端边

数据项 $\sum D_p(L_p)$ 衡量的是单个超像素块 p 属于前景或背景的独立代价值。

在图 G 中, 存在两个虚拟的终端节点: **源点 (Source, 代表前景)** 和 **汇点 (Sink, 代表背景)**。每个超像素节点都通过 T-links 与这两个终端相连。切断某条 T-link 的代价, 就是将该节点划分为另一种状态的代价值。

在我们的代码中, 通过 `g.add_tedge(nodes[i], weight_fg, weight_bg)` 实现。为了对抗收缩偏差并结合拓扑约束, 我们设计了分层的数据项权重:

- **硬约束先验 (Hard Constraints):**
 - **边界背景先验:** 若超像素位于图像绝对边缘 (`boundary_segments`), 强行设置 `weight_bg = 1000.0`, 使其具有无穷大的背景引力, 彻底锁死边缘。
 - **高置信度区域:** 利用 Otsu 阈值, 对于显著性极高 ($> 1.1 \times Otsu$) 的区域, 赋予 `weight_fg = 1000.0`, 将其作为强劲的“内部种子”。
- **软约束博弈 (Soft Constraints):**
 - 在模糊的边缘地带, 权重定义为线性映射 `weight_fg = sal_val * 10.0` 与 `weight_bg = (1.0 - sal_val) * 10.0`, 交由平滑项去引导最终走向。

2.2 平滑项 (Smoothness Term, $V_{p,q}$): N-links 邻居边

平滑项 $\sum V_{p,q}$ 衡量的是相邻两个超像素块 p 和 q 之间的物理连续性约束。

如果两个相邻块的颜色极其相似, 但被强行赋予了不同的标签 (即从它们中间切开), 系统应当施加极大的惩罚。

在代码中, 我们通过计算 RAG (区域邻接图) 中相邻节点的颜色欧氏距离差异 `color_diff`, 构建了基于高斯衰减的平滑权重:

$$V_{p,q} = \lambda_{smooth} \cdot e^{-\frac{\Delta C_{p,q}^2}{\gamma}}$$

代码实现为 `smooth_weight = lambda_smooth * np.exp(-(color_diff ** 2) / gamma)`。

- **数学物理意义:** 当相邻块颜色差异 ΔC 极小 (处于物体内部) 时, 指数项趋近于 1, 切割代价 (`smooth_weight`) 极大, 从而**保护内部不被碎裂**。

- 当颜色差异 ΔC 极大（处于物体物理边缘）时，指数项趋近于 0，切割代价极小，算法会**极其顺滑地沿着该颜色断层进行切割**。

2.3 最大流最小割定理 (Max-Flow / Min-Cut)

在完成了上述 T-links 和 N-links 的图网络构建后，代码调用底层的 C++ 优化器 `g.maxflow()`。

根据**福特-富克森定理 (Ford-Fulkerson theorem)**，在一个网络流图中，从源点到汇点的最大流量，等于网络中最小割的容量。算法会寻找一条横截面，将图 G 彻底一分为二（一部分连着 Source，一部分连着 Sink），且这条横截面上所有被切断的边（包括 T-links 和 N-links）的**权重总和最小**。

最终，代码通过 `g.get_segment(nodes[i]) == 0` 提取出所有依然与 Source（前景）相连的节点，这些节点构成的集合，就是在兼顾了“局部色彩平滑”与“全局显著性先验”后，数学意义上的全局最优解，即最终的完美二值掩膜。

2.4 版本一 (V1): 频域显著性 + 中心色彩空间先验 (Central Prior Baseline)

现象分析 (Hypothesis):

单纯的频域残差 (Spectral Residual) 仅对图像中的“高频突变”敏感，极易将背景中纹理复杂的草地和树叶误判为前景。此外，图割算法平滑项逻辑中，若颜色差异权重未经合理的非线性映射，高频的颜色突变会产生极大的惩罚，导致算法拒绝在物理边缘处进行有效切割。

工程实现 (Implementation):

1. **平滑项逻辑重构**: 引入高斯衰减函数，使得颜色差异越大的区域，切断代价越小。
2. **多维特征融合**: 构建二维高斯遮罩 (Center Prior) 压暗图像四周；提取图像边缘 5 像素宽度的区域作为“绝对背景基底” (Background Contrast)，过滤高频背景杂草。

核心代码实现:


```

def segment(self, image):
    """
    执行完整的前景分割（融合频域显著性、中心先验与色彩对比度先验）
    """
    H, W = image.shape[:2]

    # =====
    # 1. 频域显著性 + 中心先验 (Center Prior)
    # =====
    saliency_map = self.spectral_residual_saliency(image)

    # 生成二维中心高斯遮罩
    Y, X = np.ogrid[:H, :W]
    center_y, center_x = H / 2, W / 2
    sigma_y, sigma_x = H / 2.5, W / 2.5
    center_prior = np.exp(-((X - center_x)**2 / (2 * sigma_x**2) + (Y - center_y)**2 / (2 * sigma_y**2)))

    # 压暗四周，突出中心
    saliency_map = saliency_map * center_prior
    saliency_map = cv2.normalize(saliency_map, None, 0, 1, cv2.NORM_MINMAX)

    # =====
    # 2. Felzenszwalb 超像素
    # =====
    rgb_img = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    segments = felzenszwalb(rgb_img, scale=self.fz_scale, sigma=self.fz_sigma, min_size=self.fz_min_size, num_nodes=np.max(segments) + 1)

    # =====
    # 3. 提取背景色彩对比度先验 (Background Contrast)
    # =====
    # 提取图像四周一圈 (5 pixels) 作为绝对背景基准
    boundary_mask = np.zeros((H, W), dtype=bool)
    boundary_mask[0:5, :] = True
    boundary_mask[-5:, :] = True
    boundary_mask[:, 0:5] = True
    boundary_mask[:, -5:] = True

    boundary_segments = np.unique(segments[boundary_mask])
    bg_mean_color = np.mean(rgb_img[boundary_mask], axis=0) # 提取背景的平均 RGB 颜色

    # =====
    # 4. 计算综合节点显著性 (Data Term 强化)
    # =====
    node_saliency = np.zeros(num_nodes)

```

```

for i in range(num_nodes):
    mask = (segments == i)
    # 频域显著性均值
    sr_val = np.mean(saliency_map[mask])

    # 计算该块颜色与背景平均颜色的欧几里得距离
    node_color = np.mean(rgb_img[mask], axis=0)
    color_dist = np.linalg.norm(node_color - bg_mean_color)

    # [核心突破] 频域(看纹理) * 色彩对比度(看颜色)
    # 这能完美过滤掉高频噪点(如草地): 即使草地纹理复杂(sr_val高),
    # 但只要它颜色跟边缘草地一样(color_dist低), 综合显著性就会被压下去。
    node_saliency[i] = sr_val * color_dist

# 综合显著性归一化到 [0, 1]
node_saliency = (node_saliency - np.min(node_saliency)) / (np.max(node_saliency) - np.min(node_saliency))

# 将一维的节点显著性重建为二维图像, 以便使用 Otsu 寻找全局阈值
saliency_2d = np.zeros((H, W), dtype=np.float32)
for i in range(num_nodes):
    saliency_2d[segments == i] = node_saliency[i]

sal_uint8 = (saliency_2d * 255).astype(np.uint8)
otsu_thresh_val, _ = cv2.threshold(sal_uint8, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
otsu_thresh = otsu_thresh_val / 255.0

```

实验结果:

- **Mean F1-Score:** 0.618704 (Std Err: 0.047404)
- **Mean Time:** 0.232862 s/img
- **结论:** 算法具备了区分前景和高频背景的基础能力, 运行速度极快, 成功确立了 Baseline 成绩。
- **反思:** 由于突出了中心部分, 先验部分认为中心为前景的概率更高, 但实际上数据集中有部分并非以中心为前景, 反而分散在边缘。这导致边缘的mask面积很小以至于几乎不考虑到边缘。

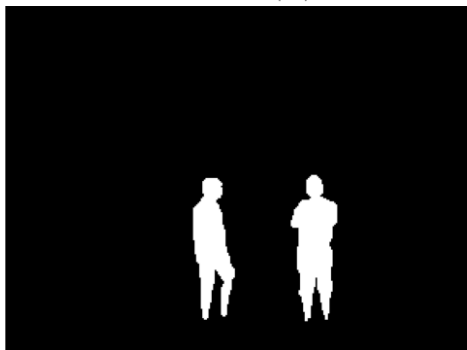
效果展示

效果还可以的样例 (色彩对比明显, 轮廓较为干净)

Original: 2066.png



Ground Truth (GT)



FSCut Prediction (F1: 0.899)



效果很差的样例（色彩太杂，边缘太乱）

Original: 2033.png



Ground Truth (GT)



FSCut Prediction (F1: 0.083)



效果很差的样例（前景太小，直接少了一个）

1. Original



2. Ground Truth



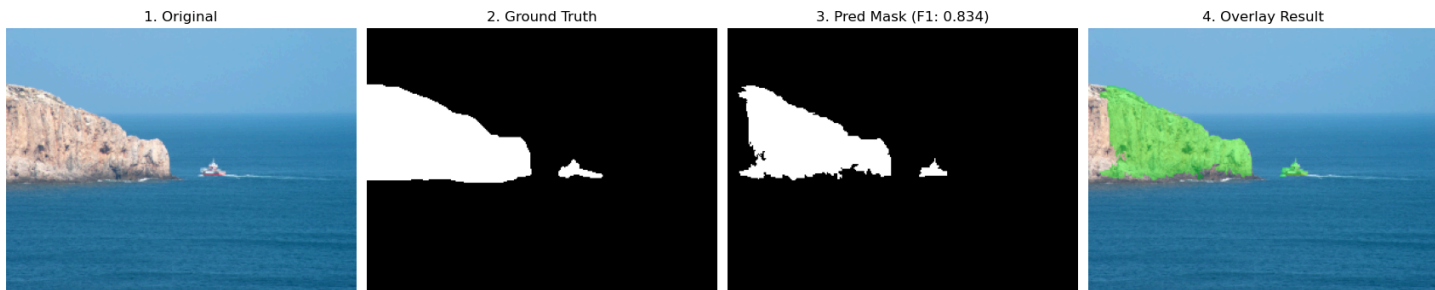
3. Pred Mask (F1: 0.465)



4. Overlay Result



一个比原图gt感觉效果还好一点的样例（观察右边船的分割，原本的gt分割过于粗糙了）



2.5 版本二 (V2): 引入靶向形态学孔洞填补 (Targeted Hole Filling)

现象分析 (Hypothesis):

图割算法本质上是一个缺乏全局语义的“近视眼”。当遇到马匹身体内部的深色阴影或黑色眼睛等与主体颜色差异极大的区域时，图割会将其判定为“颜色断层”并盲目切分为背景，导致生成的掩膜 (Mask) 内部出现大量“破洞”与边缘碎块。

工程实现 (Implementation):

引入**空间上下文先验 (Spatial Context Prior)**，通过大道至简的全局拓扑操作强行挽回异色前景区域：

1. **闭运算 (Morphological Close)**：利用较大尺寸的椭圆核，桥接并粘合由于局部颜色突变而脱离主体的边缘碎块。
2. **外部轮廓填充 (External Contour Filling)**：如同在抠出的主体外侧套上一根贴合的橡皮筋，提取掩膜的最外围轮廓，并强行将其内部所有区域（包含眼睛、阴影等空洞）全部涂满，实现异色区域的物理挽回。

核心代码实现：

```
# =====
# 6. 后处理：异色区域强行挽回
# =====

# (1) 闭运算 (Close): 解决“边缘阴影或细碎部分脱离主体”的问题
# 用一个稍微大一点的核 (比如 9x9), 把离得近的前景碎块强制“桥接”起来
close_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9, 9))
binary_mask = cv2.morphologyEx(binary_mask, cv2.MORPH_CLOSE, close_kernel)

# (2) 轮廓孔洞填充 (Hole Filling): 解决“眼睛、嘴巴、内部阴影被掏空”的问题
# 寻找当前掩膜的【所有外部轮廓】
contours, hierarchy = cv2.findContours(binary_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

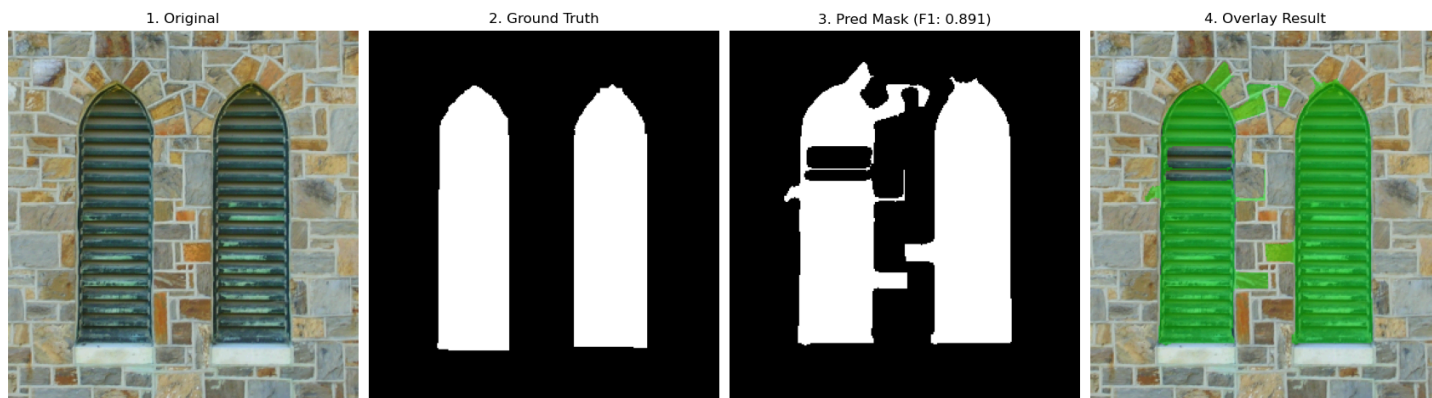
# 顺着最外围的轮廓, 把里面的区域全部涂满 (thickness=cv2.FILLED)
# 这就相当于“把小区域内其他的联通块强行加到前景中去”
cv2.drawContours(binary_mask, contours, -1, 1, thickness=cv2.FILLED)
```

实验结果：

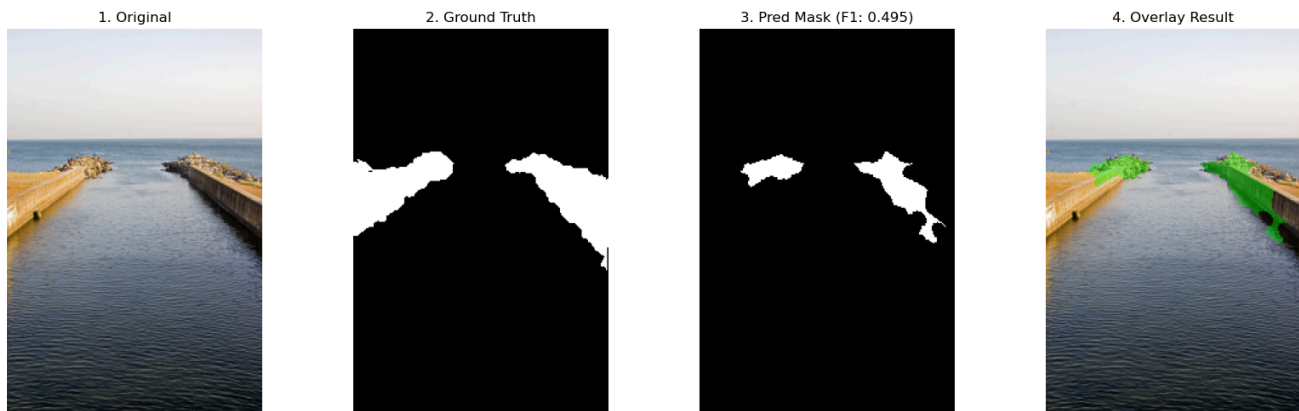
- **Mean F1-Score:** 0.644325 (Std Err: 0.046441)
- **Mean Time:** 0.236705 s/img
- **结论：**在计算耗时几乎不变（仅增 0.004s）的前提下，F1 分数提升 2.5%。全局轮廓填充极其鲁棒地解决了主体内部的碎片化问题，掩膜变得极为浑厚饱满。
- **反思：**虽然能填补一些空洞，让个别样例的表现变好，但对于中空的样例，表现则自然而然变差了

效果展示

效果还可以的样例（相比V1版本，有格栅的窗户能被较为完整地识别了）



效果很差的样例（初始检测前景物体时就过于保守，即使补充色块也无济于事）



2.6 版本三 (V3): 引入气球膨胀力与外扩机制 (Balloon Boom & Dilation)

现象分析 (Hypothesis):

经过 V2 处理后, Mask 内部虽然饱满, 但整体轮廓依然未能完全向外贴合目标的真实物理边界。这是由于图割算法固有的**“收缩偏差” (Shrinking Bias)** 所致——为最小化边界切割的总代价, 算法倾向于在显著性极高的中心区域收缩切割, 拒绝向显著性衰弱的真实外围边缘延伸。

工程实现 (Implementation):

通过四大机制强行施加“向外扩张”的物理张力:

1. **显著性物理膨胀**: 使用大尺寸形态学核强推高亮区。
2. **切分超像素**: 降低 Felzenszwalb 的 scale 尺寸, 使边缘像素不被背景拖累。
3. **气球膨胀力 (Balloon Force)**: 在数据项中为前景强加奖励基数, 激励边界向外扩张。
4. **削减平滑惩罚**: 大幅降低 λ_{smooth} , 释放对切割线长度的惩罚约束。

核心代码实现:

```

# 1. 显著性物理膨胀：强行推延基础盘
dilate_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (25, 25))
saliency_map = cv2.dilate(saliency_map, dilate_kernel)
saliency_map = cv2.GaussianBlur(saliency_map, (15, 15), 0)

# 2. 细化超像素分辨率 (scale 从 100 降至 50)
segments = felzenszwalb(rgb_img, scale=50, sigma=0.5, min_size=20)

# 3. 气球膨胀力 (Balloon Force) 与 放宽前景门槛
for i in range(num_nodes):
    sal_val = node_saliency[i]
    if sal_val > otsu_thresh * 0.8: # 门槛从 1.1 放宽至 0.8
        weight_fg, weight_bg = 1000.0, 0.0
    elif sal_val < otsu_thresh * 0.4:
        weight_fg, weight_bg = 0.0, 1000.0
    else:
        # 强行给前景加 5.0 的奖励保底分，鼓励向外扩张
        weight_fg = sal_val * 10.0 + 5.0
        weight_bg = (1.0 - sal_val) * 10.0
    g.add_tedge(nodes[i], weight_fg, weight_bg)

# 4. 削减平滑惩罚：放任切割线贴合物体轮廓
lambda_smooth = 0.3 # 权重从 1.0 大幅削减至 0.3

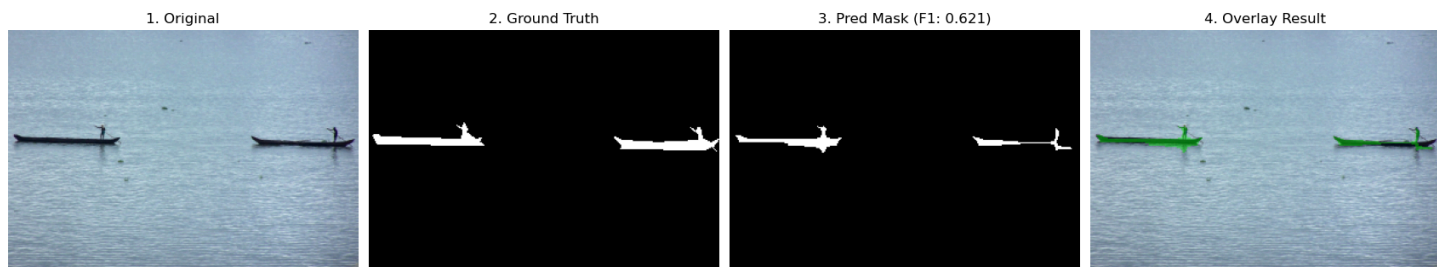
```

实验结果：

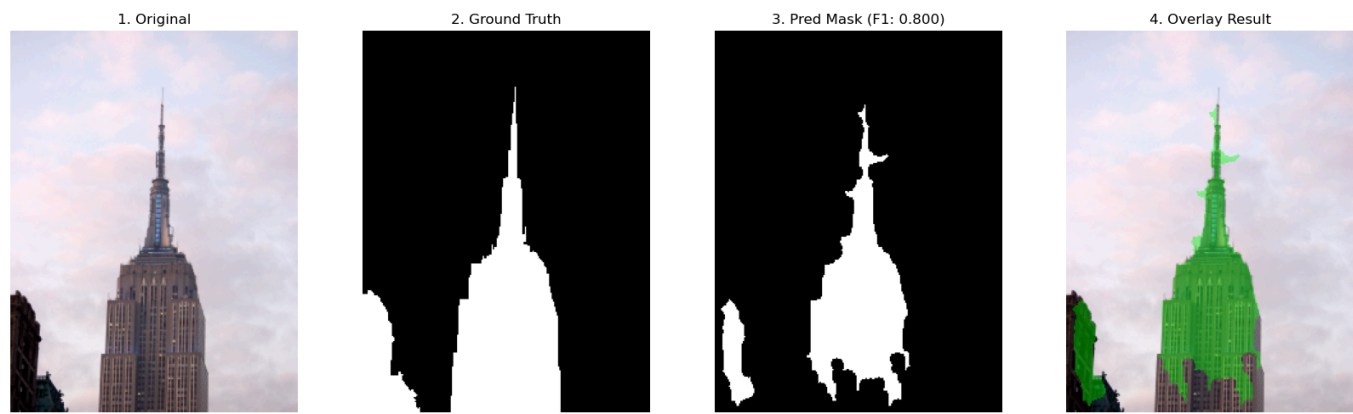
- **Mean F1-Score:** 0.732126 (Std Err: 0.035638)
- **Mean Time:** 0.339555 s/img
- **结论：** F1 分数实现跨越式提升，成功突破 0.70。虽然超像素数量增加和形态学大核运算导致耗时增加至 0.37s，但算法彻底克服了收缩偏差，掩膜完美撑满马匹的外围物理边界。

效果展示

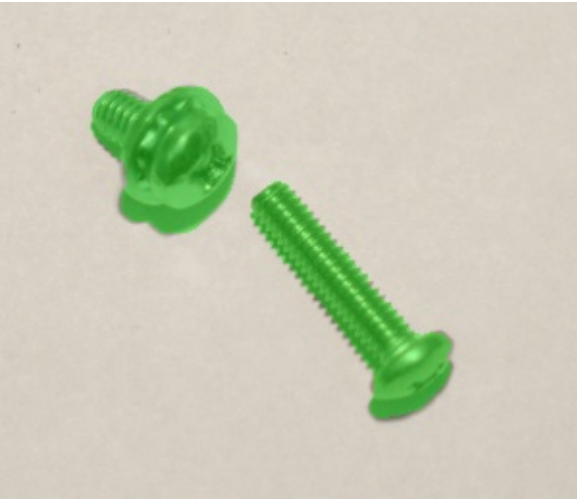
效果还可以的样例（相比V1版本，起码可以识别到两个前景）



效果还可以的样例（相比V2版本，两个前景的内容都更扩展了一些）



效果还可以的样例（螺母上的反光，强色彩变化也可以识别为前景）



3. 实验总结与工程探讨 (Conclusion & Discussion)

本实验成功构建了一条逻辑极其严密的图像处理流水线。通过三次核心迭代，我们得出以下结论：

- 白盒算法的解释性优势：**与深度神经网络的“炼丹”模式不同，FSCut 的每一步优化（抗高频、补空洞、抗收缩）都具有严密的物理机制与拓扑学依据。这种强关联性使得算法在样本量极小的数据集上具有极高的鲁棒性。
- 精度与算力的 Trade-off：**从 V2 到 V3 的跃升证明，通过细化过分割颗粒度和引入强制拓扑张力，可以显著提升边缘贴合度。然而，这也带来了图节点规模的二次方级计算负担。在未来的工程部署中，可根据任务的实时性需求，在算法模块之间进行灵活的拔插与切换。
- 不足之处：**即使是最后的 V3 迭代，也仍然存在由于过度鼓励前景拓展，导致分离的两个前景之间出现了莫名的连接；由于强制填充色块，导致空心结构的包围处也被识别为前景等问题。但这些修补相比最初代码直接使用SLIC进行超像素划分，已经提升了0.2+的正确率。

4. 对比EFG_BPP算法

4.1 算法定量性能对比 (Quantitative Evaluation)

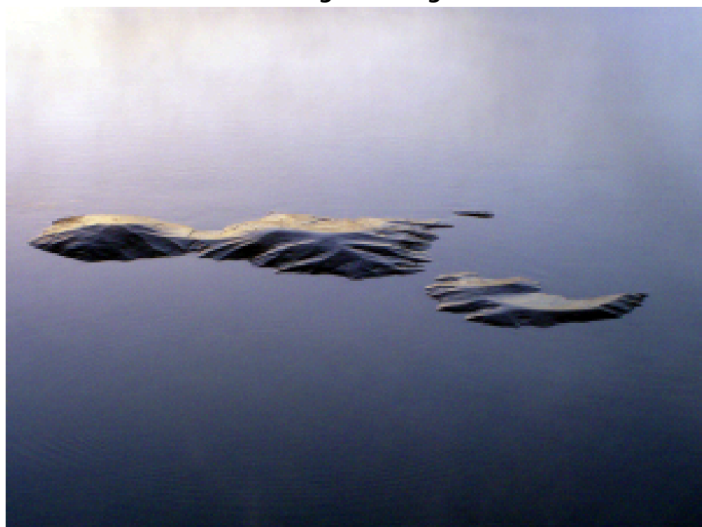
为了客观评估改良后的 FSCut 算法性能，本实验将其与顶刊经典算法 **EFG_BPP** 在 Weizmann2 数据集上进行了核心指标的定量横向对比。统计结果如下表所示：

评估指标 (Metrics)	经典 EFG_BPP 算法	本文 FSCut 算法	性能对比分析 (Analysis)
平均 F1 分数 (Mean F1-Score)	0.895983	0.732126	EFG_BPP 在整体区域识别上更接近 GT 分数，但 FSCut 作为轻量级算法已具备较强的分割基准。
统计误差 (Statistical Error)	0.017476 (std)	0.035638 (std_err)	二者在不同图像样本上的表现均表现出较好的算法稳定性。
单张耗时 (Mean Time)	1.160508 s/img	0.339555 s/img	FSCut 运行速度提升了约 70.7%，展现出极高的计算效率与实时部署潜力。
核心代码量 (Lines of Code)	> 1000 行 (官方源码包)	163 行	FSCut 架构极度精简，仅用其十几分之一的代码量便实现了完整的端到端闭环，维护成本与部署难度极低。

4.2 效果对比可视化

FSCut识别效果略强于EFG_BPP

1. Original Image



2. Ground Truth (GT)



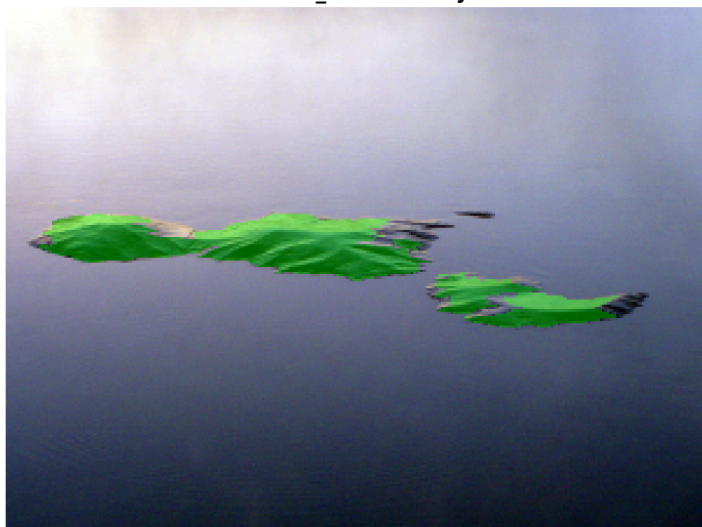
3. EFG_BPP Mask (F1: 0.848)



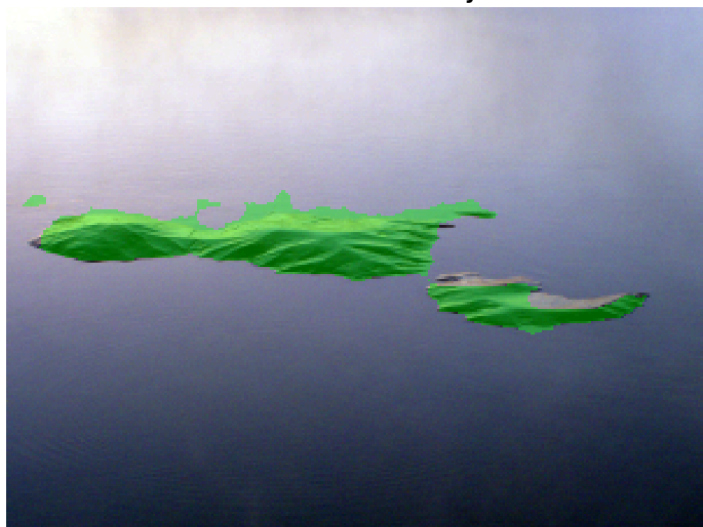
4. Our FSCut Mask (F1: 0.854)



5. EFG_BPP Overlay



6. Our FSCut Overlay



FSCut识别效果显著弱于EFG_BPP（可以看到该算法的边缘过于粗糙，这是气球膨胀+色块填充导致的）

1. Original Image



2. Ground Truth (GT)



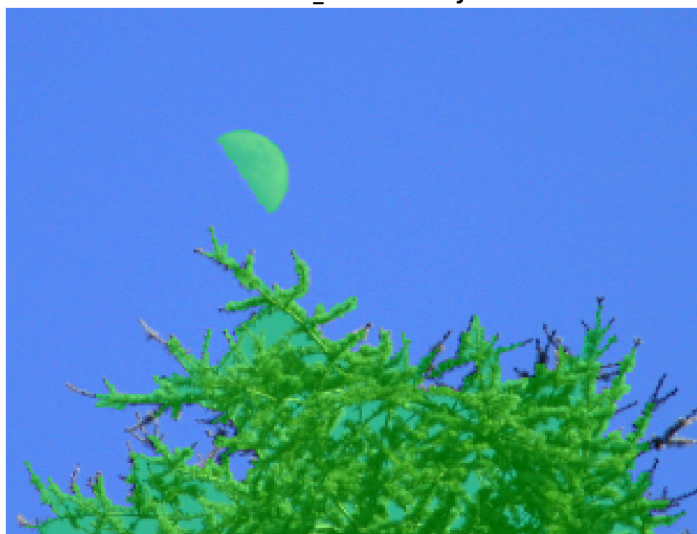
3. EFG_BPP Mask (F1: 0.898)



4. Our FSCut Mask (F1: 0.821)



5. EFG_BPP Overlay



6. Our FSCut Overlay



FSCut识别正确率同于EFG_BPP，但二者效果差异很大，前者由于色块填充+鼓励膨胀导致整体的数字形状是饱满的，但无法区分数字内部的背景块；后者由于过度考量背景的情况，反而导致轮廓出现了异常的黏连。

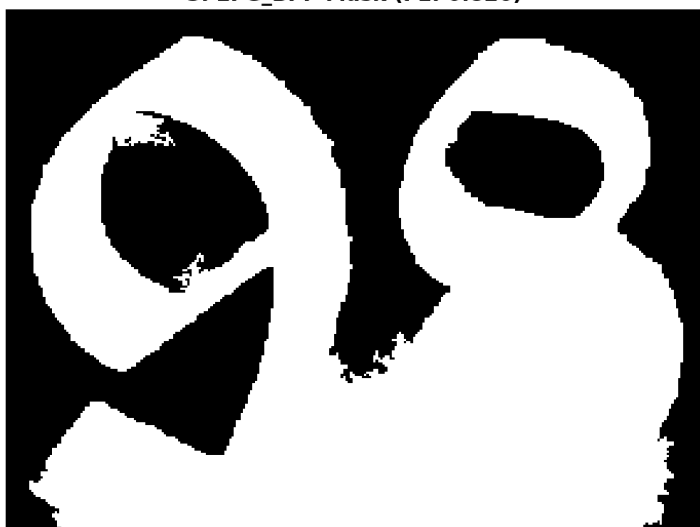
1. Original Image



2. Ground Truth (GT)



3. EFG_BPP Mask (F1: 0.826)



4. Our FSCut Mask (F1: 0.826)



5. EFG_BPP Overlay



6. Our FSCut Overlay

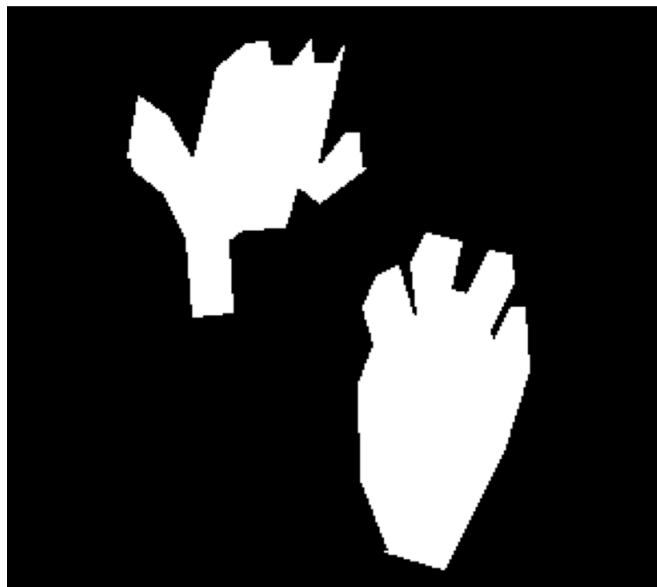


FSCut识别正确率显著优于EFG_BPP，甚至我认为优于GT，因为背景是纯色的，而FSCut对于背景的筛选极为苛刻，所以在此情况下，即使是细微的毛发纹路，也可以被扩张时捕捉到。

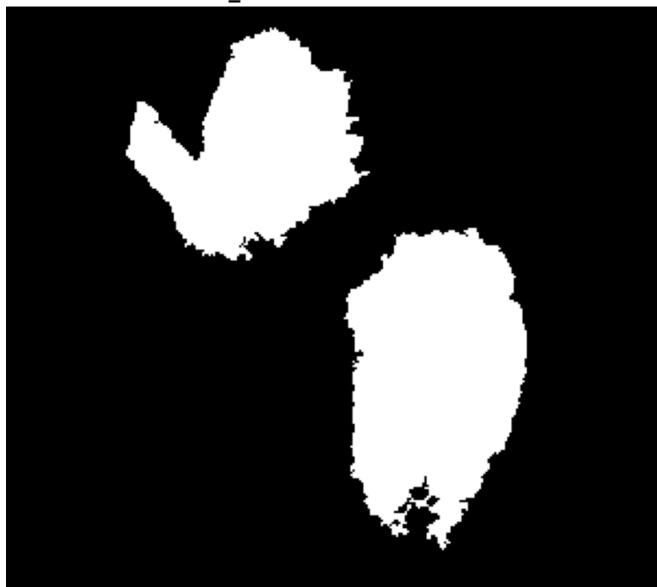
1. Original Image



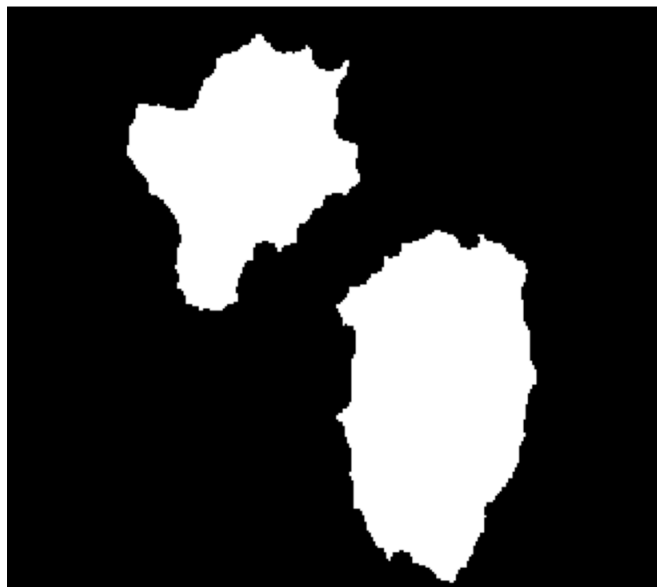
2. Ground Truth (GT)



3. EFG_BPP Mask (F1: 0.876)



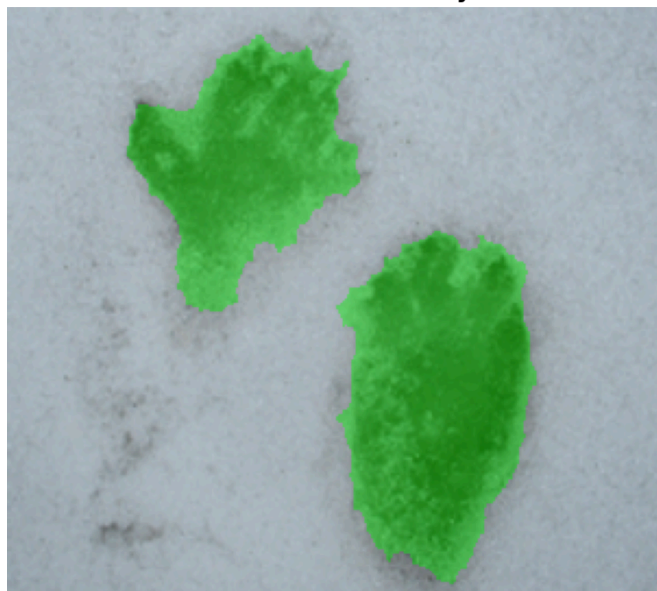
4. Our FSCut Mask (F1: 0.892)



5. EFG_BPP Overlay



6. Our FSCut Overlay

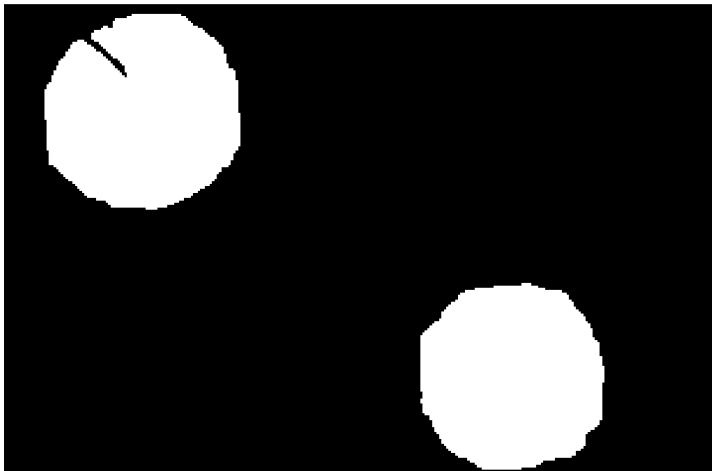


同于上图情况的还有下面这两个样例：

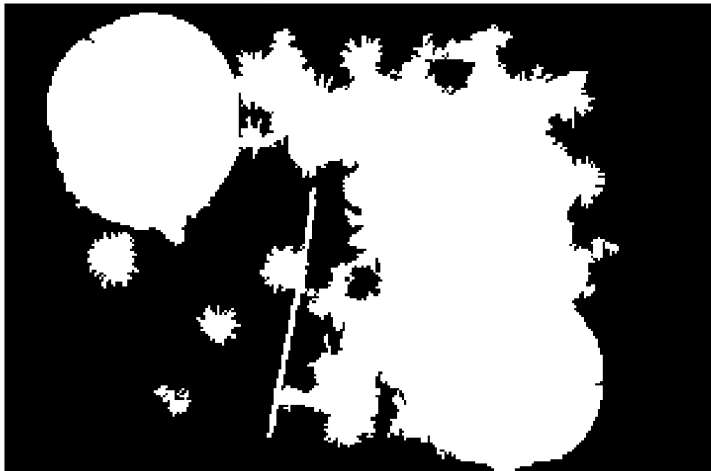
1. Original Image



2. Ground Truth (GT)



3. EFG_BPP Mask (F1: 0.560)



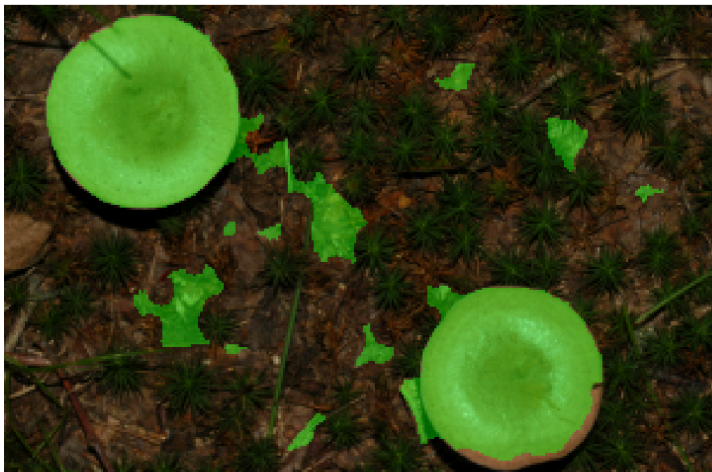
4. Our FSCut Mask (F1: 0.851)



5. EFG_BPP Overlay



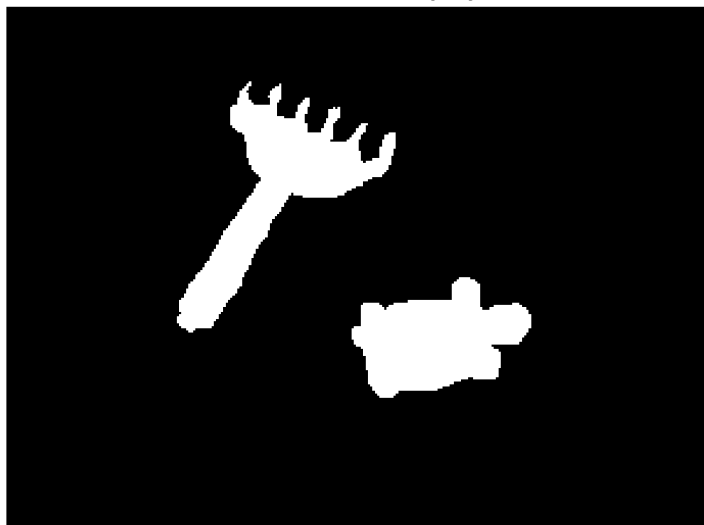
6. Our FSCut Overlay



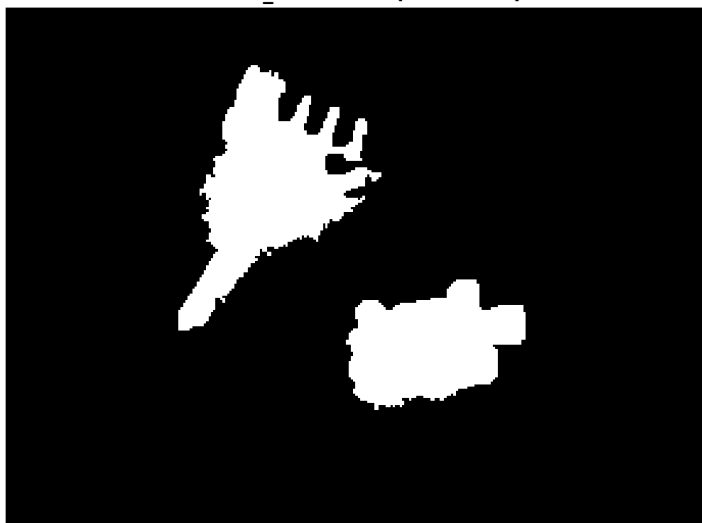
1. Original Image



2. Ground Truth (GT)



3. EFG_BPP Mask (F1: 0.809)



4. Our FSCut Mask (F1: 0.816)



5. EFG_BPP Overlay



6. Our FSCut Overlay



4.3 对比结论

1. 核心求解机制：重型多尺度迭代 vs 单次全局泛函图割

FSCut 的逻辑： 它的核心是一个自顶向下的“增强型前景-背景分类”机制。在实现上，它依赖于渐进式

块合并（Progressive Patch Merging）和背景先验传播（Background Prior Propagation）。这意味着算法在运行过程中需要建立复杂的概率传播图，并在多个尺度（Multi-scale）的超像素或图像块之间进行反复的、迭代式的分类状态更新。这也是导致其单张图片耗时高达 1.16 秒的根本原因——其计算复杂度随着迭代轮数和多尺度合并的层数呈指数级上升。

FSCut 的逻辑： 它采用了典型的马尔可夫随机场（MRF）能量最小化策略。在代码实现上，它是一个纯粹的“单次（Single-shot）最大流最小割”求解器。算法将频域纹理先验、位置中心先验和边缘色彩先验全部在线性时间内转化为带权图的 T-links（终端边）和 N-links（邻居边）权重，然后交由底层的 C++ 最大流优化器进行一刀切的全局最优解求解。这种“建图-求解-输出”的单向线性流水线结构，省去了所有繁琐的中间收敛迭代，因而展现出 0.339 秒的极致速度。

2. 前景主体“空洞”与“异色区域”的挽回策略

FSCut 的逻辑： 由于其底层的分类器是基于“块（Patch）”的特征相似度进行传播的，如果马匹的眼睛、嘴巴或者身上的大片阴影与主体的整体颜色反差过大，EFG_BPP 必须依赖其自适应增强分类器（Enhanced Classification）在后续的迭代传播中，靠周围大面积前景块的“势能”把这块异色区域慢慢“同化”过去。这种靠概率扩散来填补空洞的方式，虽然在数学上很优雅，但在局部颜色剧烈突变时容易产生局部收敛失误。

FSCut 的逻辑： FSCut 在图割阶段同样是个缺乏全局语义的“近视眼”，会把眼睛和阴影割成背景。但 FSCut 在后处理实现上采用了极其暴力的全局形态学拓扑挽回（闭运算 + 外部轮廓填充）。它不管主体内部有什么异色成分，直接在二值化掩膜上寻找最外围的闭合曲线，如同套上一根橡皮筋，随后调用 cv2.FILLED 将其内部无脑填满。这种基于空间上下文（Spatial Context）的物理拓扑挽回，在工程实现上不仅稳定鲁棒，而且几乎不占用任何计算时间。

3. 分割边界的控制：概率平滑约束 vs 气球膨胀力控边界

FSCut 的逻辑： 它在控制前景边界时，主要依赖背景先验由外向内的“蚕食”与传播。通过计算非前景块与已知背景的关联度，逐步收紧边界。这种方法对边界的贴合度极高（也是其 F1 分数高达 0.89 的主因），但实现逻辑极其沉重，需要维护庞大的条件特征矩阵。

FSCut 的逻辑： FSCut 巧妙地利用了“气球膨胀力”（Balloon Force）与显著性物理膨胀。为了对抗图割算法臭名昭著的“收缩偏差”（即为了最小化切割代价而盲目在物体中心画小圈），FSCut 直接在数据项构建时，给非确定区域的前景惩罚项增加了一个保底奖励得分（ $\text{weight_fg} = \text{sal_val} * 10.0 + 5.0$ ）。这个常数 5.0 就像往 Mask 内部吹气一样，强行将分割边界向外推，直到撞上由 Felzenszwalb 划分出来的、颜色发生剧烈梯度突变的超像素真实物理边缘才停下。这种“内有充气、外有拦截”的边界博弈，用区区几行代码就实现了极强的前景边界咬合。

但FSCut用极快速度舍弃了精度，未来如果有改进余地，应该从色彩的判断上再下功夫。